

Project Overview

Purpose of the Software

The **AI News Digest Portal** is an automated news aggregation and curation platform designed to solve the problem of information overload in the technology sector. It constantly monitors multiple global news APIs, identifies the most relevant articles related to Artificial Intelligence, and utilizes Large Language Models (Google Gemini) to read, summarize, and score the articles for importance. The final output is an easily digestible, highly relevant top-5 daily news summary formatted as an Outlook-compatible `.eml` draft.

Problem it Solves

Professionals in fast-moving industries (like AI) struggle to keep up with hundreds of daily news articles. Identifying what is actually important versus what is noise takes significant manual effort. This software automates the aggregation, filtering, reading, summarizing, and distribution of industry news, completely removing the manual curation bottleneck.

Target Users

- **Technology Executives / CTOs** needing high-level summaries of the day's most important AI news.
- **Researchers and Developers** tracking industry trends without wanting to browse multiple news sites.
- **Content Creators / Newsletter Writers** looking for a daily pre-curated list of top stories to publish.

Major Features

1. **Multi-Source Aggregation:** Pulls news from 4 independent providers (NewsAPI, MediaStack, NewsData.io, ApiTube) to ensure global coverage.
2. **AI-Powered Curation:** Uses Google Gemini to act as a human editor—reading each article, verifying its relevance, writing a concise 3-sentence summary, and assigning an Importance Score (1-10).
3. **Automated De-duplication:** Filters out duplicate stories covering the exact same event across different news outlets.
4. **Smart Dashboard:** A modern React-based interface that groups curated news by date.
5. **One-Click Digest Export:** Instantly exports the day's top 5 articles into a ready-to-send `.eml` email draft.
6. **Dynamic Prompts:** Administrators can modify the AI's core instructions via the UI to change the curation focus on the fly.

Technology Stack

- **Frontend:** React.js, Vite, TailwindCSS, Zustand (State Management), React Query, Lucide-React.
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB (via Mongoose).
- **External AI:** Google Generative AI (Gemini 1.5 Flash).
- **Authentication:** JWT (JSON Web Tokens).

High-Level Architecture

The system follows a classic decoupled client-server architecture:

- A **React Single Page Application (SPA)** handles all user interactions and polling.

- A **Node.js REST API** serves as the central brain. It receives requests from the SPA, delegates heavy background tasks (like contacting external APIs), handles MongoDB transactions, and generates files dynamically.
- **External Services** act as data feeders (News APIs) and cognitive processors (Gemini).

Design Decisions

- **Background Processing over Synchronous Loading:** News collection takes minutes due to AI rate limits. The system was designed to acknowledge the request immediately (HTTP 202) and process in the background, while the frontend polls for completion.
 - **.EML over Direct SMTP:** Initially designed to send emails automatically, the system was refactored to generate `.eml` files with `X-Unsent: 1` headers. This prevents the server from acting as an open mail relay and allows users to manually review and send the drafts in their native email clients (Outlook/Apple Mail).
 - **Zustand over Redux:** Chosen for its lightweight, boilerplate-free state management for authentication.
-

System Architecture

Core Components

The AI News Digest Portal relies on a highly modular, decoupled architecture consisting of four major components:

1. Frontend Client (React SPA)

- Serves the user interface.
- Manages localized state (Zustand) and server state caching (React Query).
- Responsible for polling background tasks and enforcing UI-level validation.

2. Backend API Server (Express.js)

- Acts as the central orchestrator.
- Protects routes via JWT Middleware.
- Delegates tasks to specific controllers and services.
- Assembles the final `.eml` exports dynamically.

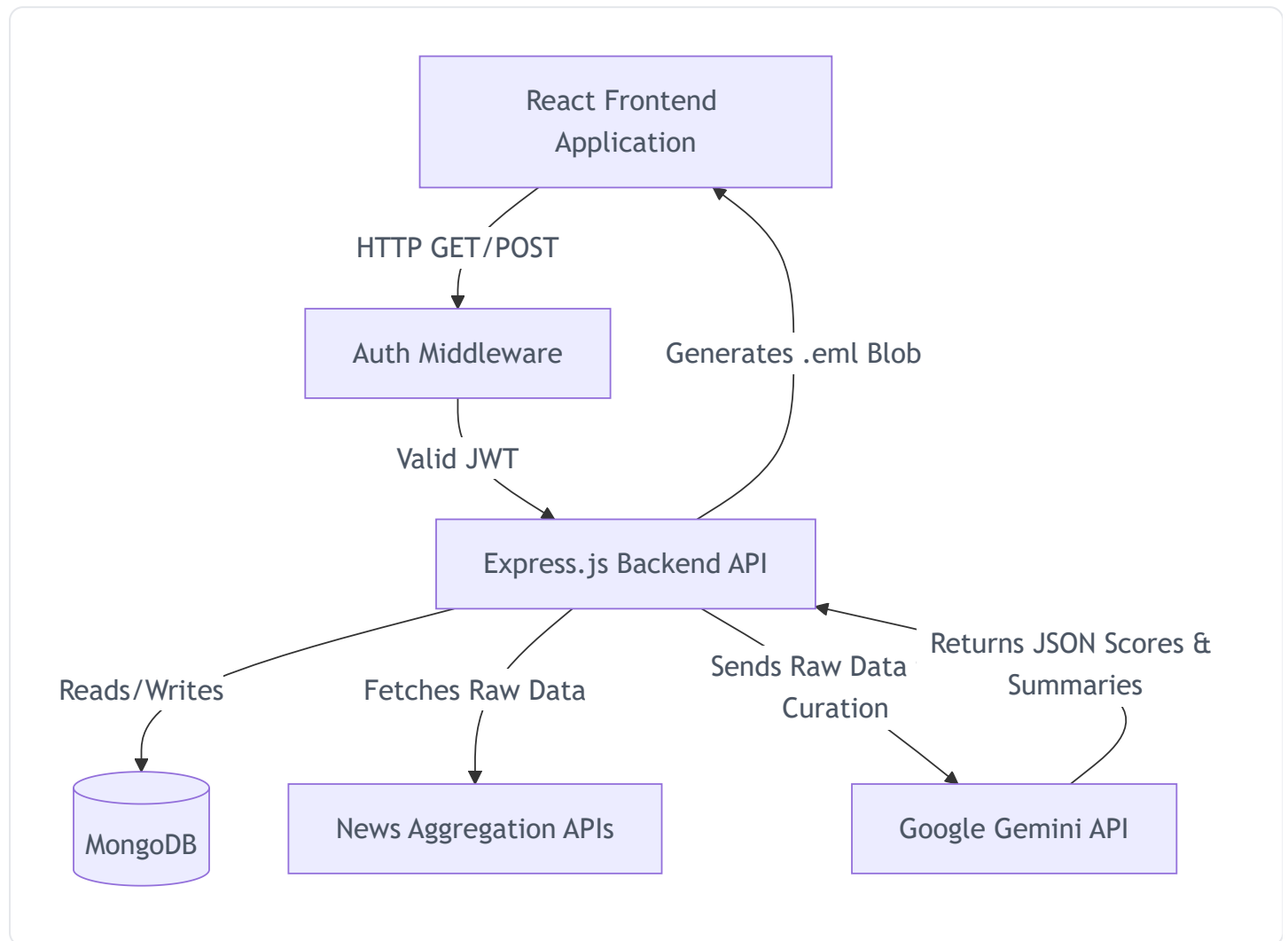
3. Database Layer (MongoDB)

- Provides persistent storage for users, AI-curated articles, settings, and historical digest logs.

4. External Services Integration Layer

- **News Providers:** REST integrations with NewsData.io, ApiTube, NewsAPI, and MediaStack to pull raw data.
- **Cognitive Engine:** REST integration with Google Generative AI (Gemini) to evaluate raw data.

Architectural Diagram



Internal Backend Architecture

The backend strictly follows the **Controller-Service-Model** pattern:

1. **Routes:** Define endpoints and apply middleware (e.g., `/api/news/collect`).
2. **Controllers:** Handle HTTP Request/Response objects, extract parameters, and return JSON or Files. They contain the main orchestration logic.
3. **Services:** Abstract out heavy external API communication. For example, `aiService.js` strictly handles formatting prompts and communicating with Google Gemini, completely isolated from Express `req/res` objects.
4. **Models:** Define the structure of data saved to the database.

Concurrency and Scaling

The most resource-intensive operation is `aiService.js`. Because Google Gemini imposes rate limits on requests, the architecture forces articles to be chunked into small batches (e.g., 5 articles per chunk). The backend iterates through these chunks sequentially rather than concurrently to prevent HTTP 429 (Too Many Requests) or 503 (Service Unavailable) errors from the Google API.

Project Setup & Requirements

To successfully run, build, and deploy the AI News Digest Portal, your development environment must meet the following hardware and software requirements.

1. System Requirements

- **Operating System:** Windows 10/11, macOS 10.15+, or Linux (Ubuntu 20.04+ recommended).
- **Memory:** Minimum 4GB RAM (8GB recommended for running both frontend and backend concurrently).

2. Core Dependencies

- **Node.js:** Version 16.x or higher (Version 18.x LTS highly recommended).
- **Package Manager:** npm (v8.x or higher) is the default package manager. Yarn can be used as a drop-in replacement.
- **Database:** MongoDB server.
 - *Option A:* Local MongoDB Community Server (v5.0+).
 - *Option B:* MongoDB Atlas (Cloud Database). This project is pre-configured to easily accept Atlas SRV connection strings.
- **Cache / Queue:** Redis Server. (Required for internal job queueing and rate limit tracking).
 - *Option A:* Local Redis instance.
 - *Option B:* Redis Enterprise Cloud / Upstash.

3. Browser Support

The React frontend leverages modern CSS and ES6 JavaScript. Ensure your browser is up to date:

- Google Chrome (latest)
- Mozilla Firefox (latest)
- Microsoft Edge (latest)
- Apple Safari (v14+)

4. IDE Recommendations

For the best developer experience, we recommend:

- **Visual Studio Code**
- Plugins:
 - ESLint
 - Prettier
 - Tailwind CSS IntelliSense
 - MongoDB for VS Code

5. Account Requirements

Before installing, you must register for the following third-party APIs to obtain keys (Detailed fully in `05_API_Keys.md`):

1. **Google AI Studio**: For Gemini API.
2. **NewsAPI.org**: For mainstream news.
3. **MediaStack**: For global news.
4. **NewsData.io**: For structured news data.
5. **ApiTube**: For supplementary technology news.

Installation Guide

Follow these steps to set up the AI News Digest Portal on your local development machine. The project is split into two distinct applications: a Node.js `backend` and a React `frontend`.

1. Backend Setup

The backend is an Express API that connects to MongoDB and Redis.

```
# Navigate to backend folder
cd backend

# Install dependencies
npm install

# Create environment file
cp .env.example .env
```

Open the `.env` file in your code editor and populate it with your actual database strings and API keys (See `04_Environment_Variables.md` for details).

Start the Backend

```
# For development (auto-restarts on file changes)
npm run dev

# For production
npm start
```

You should see a message in the terminal indicating `Server started on port 5000` and  `MongoDB Connected`.

2. Frontend Setup

The frontend is a React Single Page Application powered by Vite.

```
# Open a new terminal window/tab
# Navigate to the frontend folder from the project root
cd frontend

# Install dependencies
npm install
```

Configure Frontend API Connection

The frontend uses an Axios instance that expects the backend to run on

`http://localhost:5000/api` . If you change your backend port, you must update the base URL in `frontend/src/api/axios.js` .

Start the Frontend

```
# Run the Vite development server
npm run dev
```

The terminal will output a local link, typically `http://localhost:5173`. Open this in your browser.

3. Initial Application Boot

1. Navigate to `http://localhost:5173` in your browser.
2. Click on **Register** to create your first admin account.
3. Upon successful registration, you will be automatically logged in and redirected to the Dashboard.
4. Go to **Settings** to configure the AI Prompt and default receiver emails if desired.

Environment Variables

The backend application requires an environment file (`backend/.env`) to run. The frontend does not require any environment variables out-of-the-box as it defaults to

`http://localhost:5000/api` .

Backend `.env` File Overview

Server Configuration

- **PORT**
 - **Required:** No
 - **Default:** `5000`
 - **Example:** `5000`
 - **Purpose:** Defines the HTTP port the Express server listens on.
 - **Failure impact:** If missing, it safely falls back to `5000` .

Database Configuration

- **MONGO_URI**
 - **Required:** Yes
 - **Example:** `mongodb+srv://user:pass@cluster.mongodb.net/?appName=Cluster0`
 - **Purpose:** Defines the connection string for MongoDB.
 - **Failure impact:** The server will crash on startup if missing or invalid because `mongoose.connect()` is strictly required in `config/db.js` .

- **REDIS_HOST, REDIS_PORT, REDIS_USERNAME, REDIS_PASSWORD**
 - **Required:** Yes
 - **Example:** `REDIS_HOST=straw-crown.db.redis.io` , `REDIS_PORT=19080`
 - **Purpose:** Defines the connection to the Redis cache. Used primarily for managing background job queues (like rate-limited fetching) and session tracking.
 - **Failure impact:** Background workers (like BullMQ/Bull) will throw connection errors and news collection will fail.

Security

- **JWT_SECRET**
 - **Required:** Yes
 - **Example:** `your_super_secret_jwt_key`
 - **Purpose:** Cryptographic key used to sign and verify JSON Web Tokens for user authentication.
 - **Failure impact:** Users will be unable to log in, register, or access any protected `/api/` endpoints.

Artificial Intelligence

- **GEMINI_API_KEY**
 - **Required:** Yes
 - **Example:** `AIzaSy...`
 - **Purpose:** Authenticates requests to the Google Generative AI (Gemini) endpoint.
 - **Failure impact:** The `aiService.js` will throw HTTP 400/401 errors. News will be fetched but the application will completely fail to evaluate, score, or summarize them, resulting in 0 saved articles.

News API Providers

- **NEWSDATA_API_KEY**
 - **Required:** Yes
 - **Purpose:** Key for NewsData.io API.
- **APITUBE_API_KEY**
 - **Required:** Yes
 - **Purpose:** Key for ApiTube.io.
- **NEWSAPI_API_KEY**
 - **Required:** Yes
 - **Purpose:** Key for NewsAPI.org.
- **MEDIASTACK_API_KEY**
 - **Required:** Yes
 - **Purpose:** Key for MediaStack API.

(Note: If any single news provider key is missing, `newsController.js` will catch the failure for that specific provider and log a warning, but the overall collection process will continue using the remaining providers.)

Sample `.env.example`

Create a file named `.env` in the `backend/` directory and copy the contents below:

PORT=5000

Database

MONGO_URI=mongodb://localhost:27017/ai_news_db

Redis Config

REDIS_HOST=127.0.0.1

REDIS_PORT=6379

REDIS_USERNAME=default

REDIS_PASSWORD=

Security

JWT_SECRET=replace_me_with_a_secure_random_string

AI Keys

GEMINI_API_KEY=

News API Keys

NEWSDATA_API_KEY=

APITUBE_API_KEY=

NEWSAPI_API_KEY=

MEDIASTACK_API_KEY=

API Keys & Third-Party Integrations

The AI News Digest Portal relies on several third-party services to function. Below is the documentation for each required service.

Google Generative AI (Gemini)

- **Purpose:** Acts as the cognitive engine. Reads raw articles, evaluates their relevance to Artificial Intelligence, generates concise 3-sentence summaries, and assigns an Importance Score.
 - **How to obtain:** Visit [Google AI Studio](#), sign in with a Google account, and click "Create API key".
 - **Where to configure:** `backend/.env` -> `GEMINI_API_KEY`
 - **Permissions required:** Standard API access (default).
 - **Application usage:** Located in `backend/src/services/aiService.js`. The application sends a heavily engineered prompt alongside batches of 5 articles to the `gemini-1.5-flash` model.
-

NewsAPI.org

- **Purpose:** Fetches real-time headlines from mainstream global news outlets (e.g., TechCrunch, Wired, NYTimes).
- **How to obtain:** Register at [NewsAPI.org](#).
- **Where to configure:** `backend/.env` -> `NEWSAPI_API_KEY`
- **Permissions required:** Developer plan is sufficient for local development (limits to 100 requests/day and blocks localhost occasionally, but works for testing).

- **Application usage:** Fetched in `backend/src/controllers/newsController.js` . Queried with terms like `"artificial intelligence" OR "machine learning"` .
-

MediaStack

- **Purpose:** Secondary global news API to ensure a diverse range of sources.
 - **How to obtain:** Sign up at [MediaStack](#).
 - **Where to configure:** `backend/.env` -> `MEDIASTACK_API_KEY`
 - **Application usage:** Queried concurrently in `newsController.js` . Free tier does not support HTTPS encryption, so requests are routed via HTTP (note standard Node.js Axios behavior).
-

NewsData.io

- **Purpose:** Extracts structured technological news data, particularly from smaller tech blogs.
 - **How to obtain:** Create an account at [NewsData.io](#).
 - **Where to configure:** `backend/.env` -> `NEWSDATA_API_KEY`
 - **Application usage:** Queried concurrently in `newsController.js` . Limited to English language articles.
-

ApiTube

- **Purpose:** A final supplementary news scraper API to catch any startup or niche AI news missed by the major providers.
- **How to obtain:** Obtain a key from [ApiTube](#).
- **Where to configure:** `backend/.env` -> `APITUBE_API_KEY`

- **Application usage:** Queried concurrently in `newsController.js` .

(Note: The system was previously integrated with SendGrid (SMTP) for direct email delivery, but this was deprecated in favor of generating local `.eml` files to bypass spam filters and SMTP rate limits.)

Folder Structure

The repository is structured as a monorepo containing two distinct applications: `backend` and `frontend`. Below is a comprehensive breakdown of every significant folder and file.

```

ai-news-digest-portal/
├─ backend/
│   └─ src/
│       └─ config/
│           └─ db.js # Initializes MongoDB connection
using mongoose.
│       └─ controllers/
│           └─ authController.js # Handles login, registration, and
JWT signing.
│           └─ digestController.js # Generates .eml files from DB
records.
│           └─ newsController.js # Coordinates fetching, AI batching,
and saving.
│           └─ settingsController.js # Manages user preferences (prompts,
receiver emails).
│       └─ middlewares/
│           └─ authMiddleware.js # Validates JWT tokens and enforces
Admin roles.
│       └─ models/
│           └─ Article.js # Mongoose schema for curated news
articles.
│           └─ Digest.js # Mongoose schema logging .eml
download events.
│           └─ Settings.js # Mongoose schema for singleton app
settings.
│           └─ User.js # Mongoose schema for accounts
(includes bcrypt).
│       └─ routes/
│           └─ authRoutes.js # Routes for /api/auth/*
│           └─ digestRoutes.js # Routes for /api/digests/*
│           └─ newsRoutes.js # Routes for /api/news/* (includes
/collect and /status)
│           └─ settingsRoutes.js # Routes for /api/settings/*
│       └─ services/
│           └─ aiService.js # Google Gemini integration, prompt
building, JSON parsing.
│           └─ emailService.js # HTML template generation for the
.eml file body.
│       └─ index.js # Express server entry point, CORS
config, route mounting.
└─ .env # Secret environment variables (DO
NOT COMMIT).

```

```

|   └─ package.json                # Backend dependencies (express,
mongoose, genai).
|
└─ frontend/
|   └─ src/
|       └─ api/
|           └─ axios.js            # Centralized Axios instance with JWT
interceptors.
|       └─ assets/                # Static images and icons.
|       └─ layouts/
|           └─ MainLayout.jsx      # Sidebar navigation shell wrapping
all authenticated pages.
|       └─ pages/
|           └─ Dashboard.jsx       # Main UI: Triggers collection,
renders grouped news, auto-downloads.
|           └─ Login.jsx          # Authentication UI.
|           └─ Register.jsx       # New user onboarding.
|           └─ Settings.jsx       # UI for configuring AI prompts and
default emails.
|       └─ store/
|           └─ useAuthStore.js     # Zustand global state for tracking
current logged-in user.
|       └─ App.jsx                # React Router definition (Public vs
Protected routes).
|       └─ index.css              # Tailwind CSS directives.
|       └─ main.jsx               # React DOM entry point,
QueryClientProvider setup.
|   └─ tailwind.config.js         # Tailwind theme configurations.
|   └─ vite.config.js            # Vite build tool configuration.
|   └─ package.json              # Frontend dependencies (react,
zustand, tailwind).
|
└─ docs/                          # Project documentation (You are
here).
    └─ README.md                 # High-level entry point pointing to
the docs folder.

```

Critical Rules for File Placement

- **Never store `.env` files in source control.** They contain live credentials.
 - **Do not mix business logic with routes.** All `backend/src/routes/` files should only define HTTP paths and instantly map them to controllers.
 - **Do not put direct external API calls in controllers.** For example, Gemini API code belongs strictly in `services/aiService.js`, keeping `newsController.js` abstracted.
 - **Frontend State:** Local ephemeral state (like a modal opening) stays in the component (`useState`). Global persistent state (like the User object) belongs in `store/useAuthStore.js`. Data fetching state (loading, error, caching) belongs entirely to `@tanstack/react-query`.
-

Backend Documentation

This document explains the core files and functions within the Express.js Node backend.

1. Core Entry Point (`index.js`)

- **Purpose:** Initializes the Express application, establishes middleware (CORS, JSON parsing), connects to MongoDB via `config/db.js`, mounts API routers, and starts the HTTP server.
- **Middleware Used:** `express.json()`, `cors()`.
- **Exported Functions:** None. It is the entry script.

2. Services (`src/services/`)

`aiService.js`

- **Purpose:** Acts as a bridge between the application and Google's Gemini API.
- **Dependencies:** `@google/generative-ai`.
- **Exported Functions:**
 - `processArticlesBatchWithAI(articles, prompt, chunkIndex)`:
 - **Purpose:** Sends a chunk of raw articles to Gemini for scoring.
 - **Parameters:** `articles` (array), `prompt` (string representing system instructions), `chunkIndex` (number for logging).
 - **Return Value:** An array of objects containing `{ url, isAiRelated, summary, importanceScore, categories }`.
 - **External API Calls:** Hits `gemini-1.5-flash`.
 - **Exceptions Handled:** JSON parsing failures (retries parsing via regex if the AI wraps JSON in markdown blocks), timeouts, and 503 unavailability errors.

`emailService.js`

- **Purpose:** Formats article data into readable HTML.
- **Exported Functions:**
 - `generateEmailHTML(articles)` :
 - **Purpose:** Converts a database array of articles into a styled HTML string representing an email body.

3. Controllers (`src/controllers/`)

`newsController.js`

- **Purpose:** The largest controller handling the lifecycle of news aggregation.
- **Dependencies:** `axios`, `aiService.js`, `Article` model.
- **Exported Functions:**
 - `collectNews(req, res)` :
 - **Purpose:** Triggered by the frontend to start the background aggregation job.
 - **Execution Flow:** Immediately returns a 202 Accepted. Then internally contacts 4 news APIs. Deduplicates URLs. Batches 5 items at a time and calls `processArticlesBatchWithAI`. Evaluates the JSON, filters out `isAiRelated: false`, sorts by `importanceScore`, and inserts the top 5 into the database.
 - **Side Effects:** Modifies global state variable `isCollecting`.
 - `getCollectionStatus(req, res)` :
 - **Purpose:** Returns `{ isCollecting: boolean }` so the frontend can poll.
 - `getArticles(req, res)` :
 - **Purpose:** Returns the 100 most recent articles from the MongoDB database, sorted by `publishedAt`.

`digestController.js`

- **Purpose:** Generates `.eml` files.

- **Exported Functions:**

- `downloadDigest(req, res)` :
 - **Purpose:** Takes an array of Article IDs, converts them to HTML, wraps them in EML RFC 822 format with `X-Unsent: 1` , and forces a file download.
 - **Database Changes:** Creates a new `Digest` log record (Status: "Downloaded"). Marks the specified `Article` records as "Sent".

`authController.js`

- **Purpose:** User identity management.

- **Exported Functions:**

- `registerUser(req, res)` : Hashes passwords and creates a new `User` . Returns JWT.
- `authUser(req, res)` : Compares passwords. Returns JWT.

`settingsController.js`

- **Purpose:** Manages singleton application settings (AI Prompt).

- **Exported Functions:**

- `getSettings(req, res)` : Returns the single settings document.
- `updateSettings(req, res)` : Applies updates to the AI prompt or receiver emails.

4. Middlewares (`src/middlewares/`)

`authMiddleware.js`

- **Exported Functions:**

- `protect(req, res, next)` : Extracts the Bearer token from the `Authorization` header, verifies it using `JWT_SECRET` , looks up the user in MongoDB, and attaches it to `req.user` . Throws 401 if invalid.
 - `admin(req, res, next)` : Verifies `req.user.isAdmin === true` . Throws 401 if false.
-

Frontend Documentation

The frontend is a React SPA built with Vite.

1. Global State Management

`src/store/useAuthStore.js`

- Uses `zustand`.
- Stores the `userInfo` object containing the JWT token and user profile.
- Provides `login()` and `logout()` setter actions.
- Automatically persists to `localStorage`.

2. API Interceptor

`src/api/axios.js`

- Configures Axios with a base URL (`http://localhost:5000/api`).
- Injects an interceptor that reads the JWT from `localStorage` (via the `zustand` auth state) and attaches it as a `Bearer` token to every outgoing request.

3. Main Views (`src/pages/`)

`Dashboard.jsx`

- **Purpose:** The core workspace for viewing curated news.

- **State:**

- `isPolling` : Boolean tracking if the background news collection is active.
- `blobUrls` : Dictionary mapping date strings to local memory Blob URLs (for redownloading EMLs).

- **Hooks:**

- `useQuery` (Tanstack React Query) to fetch `/news` .
- `useEffect` to manage a `setInterval` that polls `/news/status` every 5 seconds when `isPolling` is true.

- **Interactions:**

- Groups fetched articles by date format (`MMM dd, yyyy`).
- Renders a "Download Top 5 Digest" button for every unique date group.
- Calls `downloadGroupDigest(dateStr, articles)` , which isolates the top 5 articles, asks the backend to format an `.eml` blob, and creates a hidden anchor (`<a>`) click to auto-download the file.

Settings.jsx

- **Purpose:** Admin configuration screen.
- **State:** `prompt` (String), `receiverEmails` (String array).
- **Hooks:** `useQuery` to load initial settings, `useMutation` to save them.
- **Interactions:** Allows users to tweak the AI's core behavior by editing the default system prompt, saving changes globally in MongoDB.

Login.jsx & Register.jsx

- **Purpose:** Handles authentication.
- **Interactions:** Sends email/password to `/api/auth` , receives user profile, pushes to `useAuthStore` , and navigates user to the Dashboard.

4. Layout Shell

`src/layouts/MainLayout.jsx`

- Wraps all protected routes (Dashboard, Settings).
 - Provides a persistent sidebar navigation menu utilizing `lucide-react` icons.
 - Executes `logout` and redirects to `/login`.
-

Database Documentation

The application uses MongoDB (via Mongoose ODM) to structure and persist data.

1. User Model (`models/User.js`)

- **Purpose:** Stores authentication credentials and role flags.
- **Fields:**
 - `name` (String)
 - `email` (String, Unique)
 - `password` (String)
 - `isAdmin` (Boolean, default: false)
- **Lifecycle Hooks:**
 - `pre('save')` : Hashes the password using `bcryptjs` before inserting into the database if the password field was modified.
- **Methods:**
 - `matchPassword(enteredPassword)` : Compares the plain-text password with the stored bcrypt hash.

2. Article Model (`models/Article.js`)

- **Purpose:** Stores curated news articles verified by Gemini.

- **Fields:**

- `title` (String, Required)
- `url` (String, Required, Unique): Prevents duplicate scraping.
- `source` (String)
- `publishedAt` (Date)
- `summary` (String): The 3-sentence summary generated by Gemini.
- `importanceScore` (Number): Graded by Gemini (1-10).
- `status` (String): `['Pending', 'Sent', 'Rejected']`

- **Indexes:**

- `{ status: 1, publishedAt: -1 }` : Optimizes the query used in the Dashboard to group and filter articles efficiently.

3. Digest Model (`models/Digest.js`)

- **Purpose:** An audit log of generated `.eml` files.

- **Fields:**

- `sentBy` (ObjectId, ref: 'User')
- `articles` ([ObjectId], ref: 'Article')
- `status` (String): `['Success', 'Failed', 'Downloaded']`

- **Relationships:** A Digest contains many Articles.

4. Settings Model (`models/Settings.js`)

- **Purpose:** A strict singleton document storing global application configurations.

- **Fields:**

- `defaultPrompt` (String): The exact prompt sent to Gemini AI.
- `receiverEmails` ([String]): Array of target emails injected into the `.eml` draft.

- **Lifecycle:** Seeded automatically at server startup if it doesn't exist. There will only ever be a single document in this collection.

API Routes Documentation

All API routes are prefixed with `/api`.

Authentication Routes (`/api/auth`)

POST `/api/auth/register`

- **Purpose:** Creates a new user.
- **Request Body:** `{ name, email, password }`
- **Response:** `201 Created` with `{ _id, name, email, isAdmin, token }`
- **Middleware:** None.

POST `/api/auth/login`

- **Purpose:** Authenticates a user and returns a JWT.
 - **Request Body:** `{ email, password }`
 - **Response:** `200 OK` with `{ _id, name, email, isAdmin, token }`
 - **Errors:** `401 Unauthorized` if invalid email or password.
-

News Routes (`/api/news`)

POST `/api/news/collect`

- **Purpose:** Triggers the background data gathering and AI curation task.
- **Middleware:** `protect`, `admin`

- **Response:** 202 Accepted with { message: "News collection started in background" }
- **Side effects:** Triggers heavy HTTP requests to third-party news APIs and Gemini.

GET /api/news/status

- **Purpose:** Polling endpoint for the frontend to check if the collection job is running.
- **Middleware:** protect , admin
- **Response:** 200 OK with { isCollecting: boolean }

GET /api/news

- **Purpose:** Retrieves curated news from the database.
- **Query Params:** limit (default: 20), timeframe (optional).
- **Middleware:** protect
- **Response:** 200 OK with { articles: [...], page, pages, total }

Digest Routes (/api/digests)

POST /api/digests/download

- **Purpose:** Generates and downloads the .eml draft containing the provided articles.
- **Request Body:** { articleIds: ["id1", "id2"] }
- **Middleware:** protect , admin
- **Response:** 200 OK Binary Blob (RFC 822 format / message/rfc822) with Content-Disposition: attachment .
- **Errors:** 400 Bad Request if no IDs provided.

GET /api/digests

- **Purpose:** Retrieves history of generated digests.
 - **Middleware:** `protect` , `admin`
 - **Response:** `200 OK` with `[ { _id, sentAt, articles: [], status } ]`
-

Settings Routes (/api/settings)

GET /api/settings

- **Purpose:** Fetch global application settings.
- **Middleware:** `protect` , `admin`
- **Response:** `200 OK` with `{ defaultPrompt, receiverEmails }`

PUT /api/settings

- **Purpose:** Updates the settings singleton document.
 - **Request Body:** `{ prompt, receiverEmails }`
 - **Middleware:** `protect` , `admin`
 - **Response:** `200 OK` with updated document.
-

Authentication & Authorization

The application uses JSON Web Tokens (JWT) for stateless authentication.

Login Flow

1. User enters credentials on `Login.jsx`.
2. Frontend sends a `POST` request to `/api/auth/login`.
3. Backend controller (`authController.js`) searches for the user by email in MongoDB.
4. Backend uses the `matchPassword` method (which uses `bcryptjs`) to verify the hashed password.
5. If successful, the backend generates a signed JWT using `JWT_SECRET` (valid for 30 days) and returns it alongside the user profile.
6. The frontend receives the response. Zustand (`useAuthStore`) saves the profile and token into the browser's `localStorage`.
7. React Router automatically detects the token in Zustand state and redirects the user from the `/login` route to the protected `/dashboard` route.

Logout Flow

1. User clicks Logout in the sidebar.
2. Zustand `logout()` action is fired.
3. The token is deleted from `localStorage` and memory.
4. React Router detects the missing token and instantly redirects to `/login`.
5. No backend call is required because JWTs are stateless.

Route Protection (Authorization Middleware)

`authMiddleware.js` contains two crucial functions to protect API endpoints from unauthorized access:

- **protect:** Intercepts requests, extracts the `Authorization: Bearer <token>` header, and uses `jwt.verify` to decode it. It then queries the database for the embedded user ID, strips the password from the object, and attaches the user to the `req` object (`req.user = await User.findById(decoded.id).select('-password')`).
- **admin:** Checks if `req.user.isAdmin === true` .

If a protected route (like generating a digest) is accessed without a valid token, the server strictly returns `401 Unauthorized` and aborts the request.

Business Logic & Workflows

The main purpose of the application is to automate the extreme manual effort required to find, read, evaluate, and format news. Below is the detailed orchestration of the primary workflow.

The Core Process: "News Collection"

1. Trigger

- User clicks **"Fetch Latest News"** on the Dashboard.
- Frontend sets `isPolling = true` and shows a spinning loader.
- A `POST` request is sent to `/api/news/collect`.

2. Backend Aggregation (The Feeders)

- The backend controller (`newsController.js`) receives the request. It returns a `202 Accepted` immediately so the browser does not hang.
- It fires off parallel asynchronous requests to 4 independent APIs (NewsAPI, MediaStack, NewsData, ApiTube).
- It requests articles matching keywords like `"artificial intelligence"` OR `"machine learning"`.
- The APIs return massive arrays of unverified, noisy data.

3. Data Cleansing

- The backend flattens the arrays into one giant list.
- It maps over them to standardize the schema (handling differences between how NewsAPI formats a title vs how MediaStack does).
- It removes articles with missing URLs or titles.
- It strictly filters out known "junk" domains (e.g., `yahoo.com` , `finance.yahoo.com` , `globoNewsWire.com`) which typically spam press releases rather than journalism.
- It de-duplicates the list based on the exact URL.

4. AI Curation (The Brain)

- The clean list (often 100+ articles) is sliced into small "chunks" (5 articles per chunk).
- *Why chunks?* Google Gemini has strict rate limits. Sending 100 articles at once would result in a massive prompt that exceeds token limits or causes the AI to hallucinate.
- For each chunk, the backend constructs a massive string combining the global AI Prompt (from `Settings`) and the text/URLs of the 5 articles.
- It calls `gemini-1.5-flash` via `aiService.js` , explicitly instructing the model to reply **only in JSON format**.
- The AI evaluates each article, writes a summary, checks if it's truly AI-related (ignoring articles about video game AI bots, for example), and grades it out of 10.
- If a Gemini request fails (e.g. 503 Overloaded), the system logs the error and gracefully skips the chunk, moving to the next one to ensure the overall job completes.

5. Final Database Storage

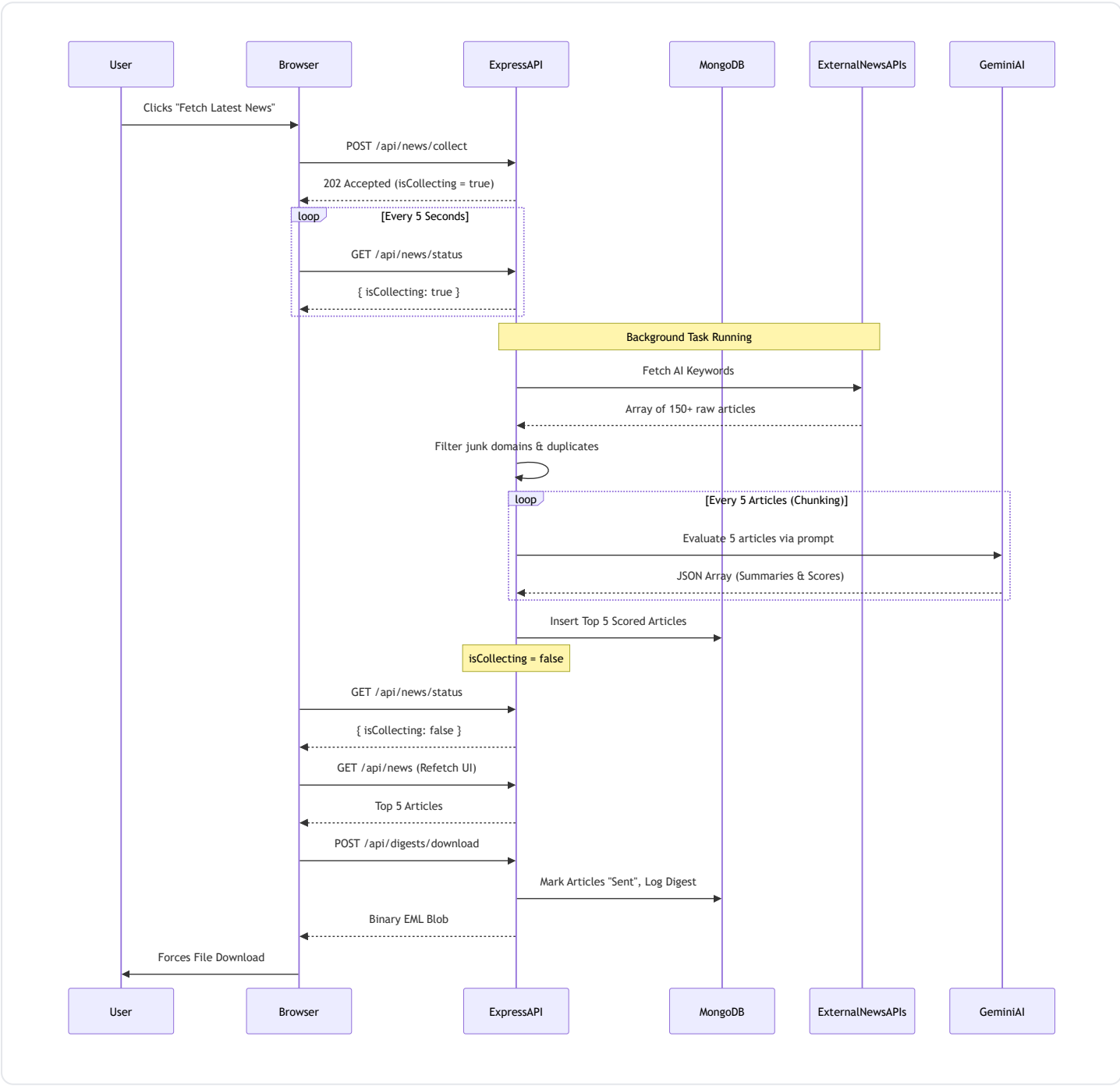
- Once all chunks are processed, the backend has an array of verified JSON results.
- It filters out any where `isAiRelated` is false.
- It sorts the remainder by `importanceScore` (descending).
- It takes the absolute top 5 best articles from the entire run and inserts them into the MongoDB `Article` collection with a status of `Pending` .
- The global `isCollecting` variable is flipped to `false` .

6. Frontend Detection & Auto-Download

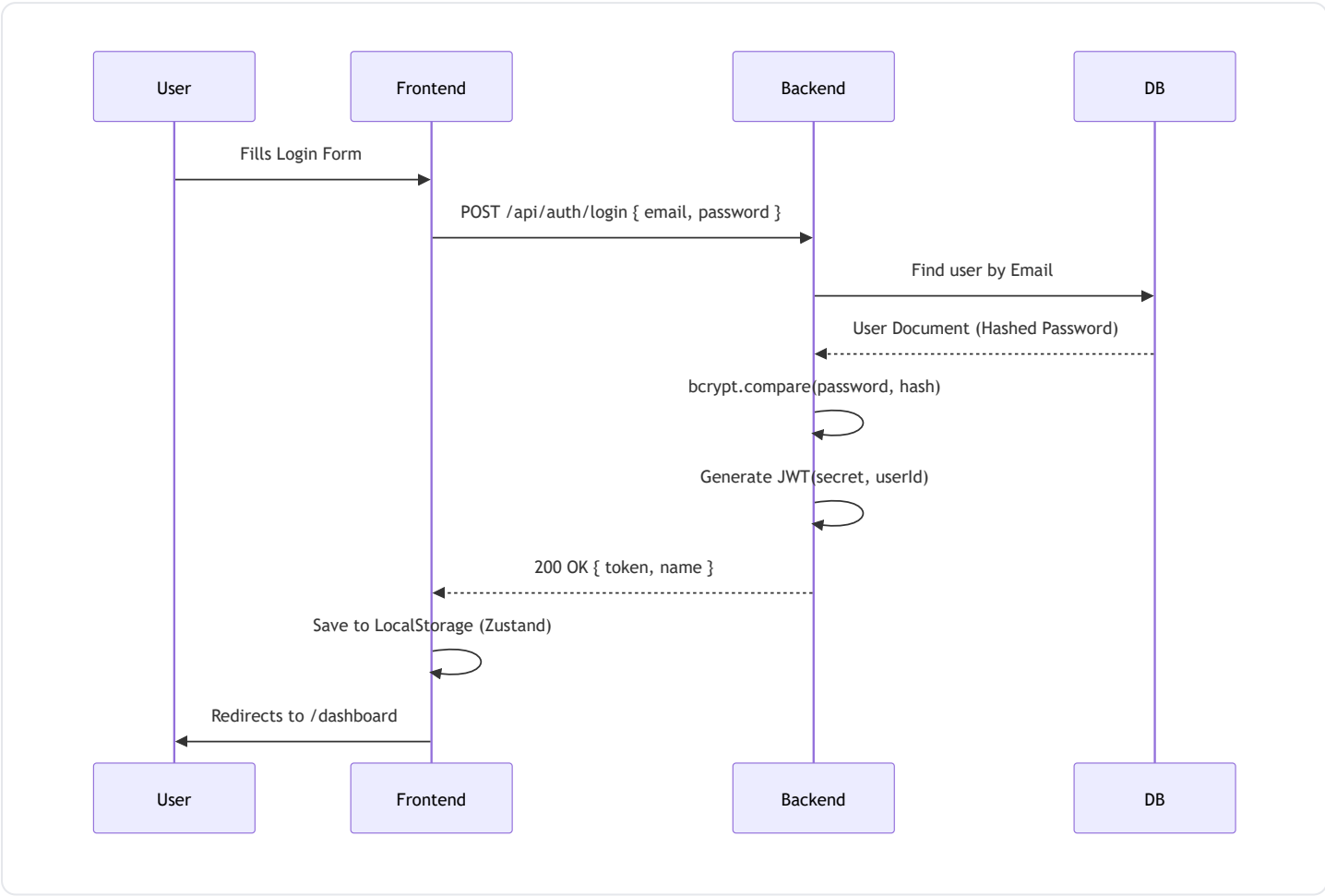
- The frontend's `setInterval` polling hits `/api/news/status` and notices `isCollecting` is now false.
 - The frontend immediately refetches the `Articles` endpoint.
 - It finds the newly fetched articles matching today's date.
 - It automatically fires a `POST` request to `/api/digests/download` with the top 5 article IDs.
 - The backend wraps them in `.eml` format, marks them `Sent`, and logs a `Digest` record.
 - The frontend forces the browser to download the file to the user's computer, completely automating the curation-to-distribution pipeline.
-

Sequence Workflows (Mermaid)

Application Polling & Curation Lifecycle



Authentication Lifecycle



External Services Integration

Google Generative AI (Gemini)

The application relies heavily on `gemini-1.5-flash` to evaluate the quality of news.

- **Integration Point:** `backend/src/services/aiService.js`
- **Rate Limit Handling:** The Google API is notoriously strict with free-tier rate limits (both Requests-Per-Minute and Tokens-Per-Minute).
- **Implementation Strategy:** The backend implements "Chunking". It slices the array of 150+ raw articles into chunks of 5. It uses a `try/catch` block for each chunk. If a chunk hits a `503 Service Unavailable` or `429 Too Many Requests`, the code explicitly catches it, logs a warning (`Error processing batch: 503`), and skips that specific batch, ensuring the system does not crash and continues processing the rest of the chunks.

News APIs

- **NewsAPI.org:** Excellent for top-tier journalism. However, their Developer Tier blocks `CORS` requests directly from a browser and occasionally blocks generic `localhost` server requests. The backend circumvents browser CORS by acting as a proxy server.
- **MediaStack:** Provides a massive volume of global news. The free tier does not support HTTPS `https://api.mediastack.com`. To prevent Axios from failing on secure environments, the endpoint is strictly hardcoded to `http://api.mediastack.com` within `newsController.js`.
- **NewsData.io & ApiTube:** Used to fill gaps and find niche AI startups that mainstream media ignores.

External Resiliency

If any single news provider goes down or rotates its API keys, the system is designed to catch that specific provider's failure via `Promise.allSettled()` (conceptually) rather than crashing the entire collection loop. The AI will simply curate from the providers that successfully responded.

Deployment Guide

To move this application from a local development environment to a production server (like AWS, DigitalOcean, or Heroku), follow these steps.

1. Frontend Build (React)

The frontend must be compiled from raw JSX into static HTML/CSS/JS.

```
cd frontend
npm run build
```

This command generates a `dist/` folder. This folder contains entirely static assets. You can host this folder cheaply or freely on services like:

- **Vercel**
- **Netlify**
- **AWS S3 + CloudFront**
- **Nginx** (If hosting on your own VPS).

Important: Before building for production, ensure you update `axios.js` or your Vite `.env.production` file so that the API base URL points to your actual backend domain (e.g., `https://api.yourdomain.com/api`) instead of `http://localhost:5000/api` .

2. Backend Deployment (Node.js)

The backend requires a persistent Node environment.

Using PM2 (On a VPS like DigitalOcean or AWS EC2)

Never use `npm run dev` (nodemon) in production. It wastes memory.

1. SSH into your server.
2. Clone the repository and run `npm install` inside the `backend` folder.
3. Install PM2 globally: `npm install -g pm2`
4. Start the server: `pm2 start src/index.js --name "ai-news-api"`
5. Save the process list so it survives server reboots: `pm2 save` and `pm2 startup`.

Reverse Proxy (Nginx)

To expose your Node app on port 80/443, configure Nginx to reverse proxy traffic from `https://api.yourdomain.com` to `http://localhost:5000`. Always secure your production API with an SSL certificate using Let's Encrypt (`certbot`).

3. Database (MongoDB Atlas)

Do not host your own MongoDB instance unless you have a dedicated DevOps team.

1. Create a free cluster on [MongoDB Atlas](#).
 2. Allow IP access from your backend production server's IP address (or `0.0.0.0/0` if on a dynamic host like Heroku, though less secure).
 3. Copy the SRV Connection String and set it as the `MONGO_URI` in your production server's environment variables.
-

Error Handling & Recovery

Controller-Level Handling

Every Express controller is wrapped in standard `try/catch` blocks. When an asynchronous error occurs, it is caught and handled via a `res.status(500).json({ message: '...', error: error.message })` response.

Axios Error Interception (Frontend)

The centralized `axios.js` file intercepts all responses from the backend. If a `401 Unauthorized` is detected (meaning a JWT has expired or is invalid), the frontend could optionally trigger a global logout event, forcing the user back to the login screen. Otherwise, local components handle the errors using React Query's `isError` and `error` states to display beautiful UI alerts (via Tailwind).

AI Fallback & Recovery Mechanisms

Google Gemini throws errors frequently when overwhelmed (503) or rate-limited (429). In `aiService.js` and `newsController.js`:

- If the AI returns JSON wrapped in markdown tags (e.g., ````json [...] ````), the service implements a Regex recovery block (`text.replace(/```json|```/g, '')`) to extract and parse the pure JSON safely.
- If the AI throws a hard 503 error, the catch block in the `for...of` chunking loop intentionally does **not** re-throw the error. Re-throwing would crash the entire collection loop. Instead, it logs the failure for that specific 5-article batch and gracefully `continue` s to the next batch. This guarantees that at least some news will be curated even during heavy API turbulence.

Database Validation

Mongoose Enums enforce strict data boundaries. For example, the `Digest` model strictly requires `status` to be `'Success'`, `'Failed'`, or `'Downloaded'`. If the controller attempts to pass an invalid string, Mongoose rejects the write at the database level and throws a `ValidationError`, which is safely returned as a 500 error to the client.

Logging & Tracing

Currently, the application relies on standard `console.log` and `console.error` for outputting telemetry to the backend terminal.

Production Logging Best Practices

For production environments, replacing `console.log` with a structured logger like **Winston** or **Pino** is highly recommended.

Key Logging Locations

- **newsController.js**: Logs the exact number of articles successfully retrieved from each individual provider. Useful for identifying if a specific API key has expired.
- **aiService.js**: Logs the batch chunk numbers as they are sent to Google Gemini, allowing developers to track exactly how far along the curation process is in real-time.
- **index.js**: Logs database connection successes and server port bindings.

Debugging Approach

If articles are not appearing in the Dashboard:

1. Check the Node.js terminal output.
 2. Look for `Error processing batch: 503`. This indicates Google Gemini is overloaded.
 3. Look for Mongoose validation errors. This indicates the AI returned a JSON schema that did not match the expected Article schema (e.g. returning a string instead of a number for the Importance Score).
-

Security Architecture

1. Password Storage (bcryptjs)

Passwords are never stored in plain text. When a user registers or changes their password, a Mongoose `pre('save')` hook intercepts the document, generates a secure random salt, and hashes the password using `bcryptjs`. Any breach of the database will only reveal irreversible hashes.

2. Authentication Tokens (JWT)

JSON Web Tokens are used. They are cryptographically signed by the backend using the highly secretive `JWT_SECRET` environment variable. If a malicious actor alters a token payload, the cryptographic signature will fail validation in the `authMiddleware.js`, throwing a 401 error.

3. Cross-Origin Resource Sharing (CORS)

The backend explicitly enables CORS via the `cors()` middleware in `index.js`. In production, this should be tightly restricted to only allow requests originating from the production frontend domain (e.g., `https://frontend-domain.com`).

4. API Key Concealment

None of the external API keys (Gemini, NewsAPI, etc.) are ever exposed to the React frontend. The frontend simply asks the backend to "fetch news", and the backend securely applies its environment variables server-side before contacting third parties.

5. File Download Security

Browsers may flag `.eml` downloads dynamically generated via `blob:` URLs as "potentially unsafe". This is a standard browser heuristic against drive-by-downloads, and since the file is constructed locally by the user's trusted backend, it is 100% safe to retain.

Troubleshooting Guide

1. MongoDB Connection Refused (`MongoNetworkError`)

Symptom: Server crashes immediately on startup. **Cause:** The `MONGO_URI` in `.env` is missing, invalid, or your IP address is not whitelisted in MongoDB Atlas. **Fix:** Verify your `.env` file. If using Atlas, log into the Atlas dashboard, navigate to "Network Access", and add your current IP address (or `0.0.0.0/0` for universal access).

2. "Error processing batch: 503" or "429 Too Many Requests"

Symptom: The backend terminal logs these errors during News Collection, and very few articles are saved to the dashboard. **Cause:** Google's Gemini API is actively rate-limiting your account. **Fix:** Wait a few minutes before clicking "Fetch Latest News" again. If this is a persistent issue in production, you must upgrade your Google AI Studio account to a paid tier to increase your Requests-Per-Minute quota.

3. Empty Dashboard After "Fetch Latest News" Completes

Symptom: The UI spinning wheel stops, but no new articles appear for today. **Cause:** The AI rejected all fetched articles because none of them met the relevance criteria outlined in the Global Prompt (Settings). **Fix:** Navigate to Settings and ensure the AI Prompt is not overly restrictive.

4. Automatic Download Fails / Browser Security Warning

Symptom: The browser flashes a warning saying "This file may be dangerous" when downloading the `.eml` file, or blocks the download entirely. **Cause:** Browsers inherently distrust `.eml` files generated automatically via `blob:` URLs without direct user clicks. **Fix:** Instruct users to click "Keep" on the warning, as the file is locally and safely generated. Alternatively, users can manually click the "Redownload Digest" button.

5. Cannot Login (401 Unauthorized)

Symptom: Login fails despite entering correct credentials. **Cause:** `JWT_SECRET` may have been changed in the `.env` file, invalidating all previously issued tokens, or the user's account does not exist in the current database instance. **Fix:** Register a new account.

Developer Guide

This guide explains how to safely modify and extend the AI News Digest Portal.

Adding a New News API Source

If you want to add a 5th news provider to the aggregation pipeline:

1. Open `backend/src/controllers/newsController.js`.
2. Locate the `collectNews` function.
3. Add a new `try/catch` block that makes an Axios request to your new provider.
4. Format the incoming data to match the standard expected format:

```
const myNewArticles = response.data.articles.map(article => ({
  title: article.title,
  url: article.link,
  source: 'MyNewProvider'
}));
```

5. Push `myNewArticles` into the master `allArticles` array.
6. The existing deduplication and AI chunking logic will automatically handle the rest.

Modifying the Database Schema

If you want to add a new field (e.g., `author`) to the curated articles:

1. Open `backend/src/models/Article.js`.
2. Add `author: { type: String }` to the schema.
3. Open `backend/src/services/aiService.js`.

4. Modify the `response_schema` object sent to Google Gemini to explicitly instruct the AI to extract and return an `author` field.
5. Update the frontend `Dashboard.jsx` to display `article.author` .

Extending the EML Export Format

If you want to change how the `.eml` file looks when opened in Outlook:

1. Open `backend/src/services/emailService.js` .
2. Modify the `generateEmailHTML` function. This function returns raw HTML. You can inject custom CSS styles (must be inline or in a `<style>` block; external stylesheets do not work reliably in email clients).
3. If you want to add an attachment to the `.eml` , you must modify `digestController.js` to implement MIME multipart boundaries (this is highly complex and requires understanding RFC 822 standards).

Adding a New Frontend Page

1. Create `src/pages/MyNewPage.jsx` .
 2. Open `src/App.jsx` .
 3. Import the page and add a `<Route path="/mynewpage" element={<MyNewPage />} />` inside the `<Route element={<MainLayout />}>` block to ensure it is protected by authentication and wrapped in the sidebar.
 4. Add a navigation link to `src/layouts/MainLayout.jsx` using `lucide-react` icons.
-